

Term Project: Big Data

1. Overview & Learning Objectives

This term project asks you to build a complete analytics pipeline for a fictional mid-size online retailer, “UrbanCart,” moving from raw, imperfect data all the way to a decision-ready report. It is intentionally designed so that SQL, NumPy, and pandas are not optional add-ons but load-bearing parts of the workflow; you cannot finish the project well using only one of the three.

2. Business Scenario: UrbanCart

UrbanCart is a fictional online retailer selling six product categories (Electronics, Home & Kitchen, Apparel, Sports & Outdoors, Beauty & Health, Books & Media) across fifteen cities worldwide. UrbanCart’s leadership team has asked you – acting as their newly hired data analyst – to answer three big questions:

1. Who are our most valuable customers, and what drives their loyalty (or churn)?
2. Which products and categories are actually profitable once returns, discounts, and cost are accounted for – and can we forecast near-term demand?
3. Is our marketing/signup data (spread across an old CRM export and the current database) trustworthy enough to make decisions on, and if not, how should it be fixed?

Your final report is the deliverable leadership will read. Treat the dataset as “real”: it contains the kinds of errors, duplicates, and inconsistencies that show up in actual company data, and part of your job is noticing and handling them – not assuming the data is clean.

3. Dataset & Schema

You will be provided with a SQLite database, *ecommerce.db*, plus two external CSV files that are deliberately NOT already integrated into the database. Reconciling them is part of the assignment. A full data dictionary is included in *README_DATA.md* alongside the files.

3.1 Database tables (ecommerce.db)

Table	Key Columns	Purpose / Known Issues
customers	customer_id (PK), email, signup_date, city, age, gender	~3% missing city, ~6% missing age
products	product_id (PK), category, subcategory, unit_price, cost	~1% extreme price outliers
orders	order_id (PK), customer_id (FK), order_date, status, payment_method	status includes completed / returned / cancelled / pending
order_items	order_item_id (PK), order_id (FK), product_id (FK), quantity, unit_price, discount	negative quantity = return; ~0.8% exact duplicate rows
reviews	review_id (PK), product_id (FK), customer_id (FK), rating, review_text	a few out-of-range ratings; ~20% missing text
web_sessions	session_id (PK), customer_id (FK), session_date, device, duration_minutes	for engagement / time-series analysis

3.2 External files (require pandas-based integration)

File	What makes it messy
legacy_customers_export.csv	Four different date formats across rows, inconsistent name casing, a header with trailing whitespace, ~5% duplicate/near-duplicate customers, missing emails, and junk rows (test accounts, blank rows).
product_catalog_2024.csv	Different column names/casing than the products table (SKU vs product_id, dept vs category), partial coverage overlap, and a dozen supplier-only SKUs absent from the database entirely.

4. Project Requirements

4.1 Phase 1 – SQL Data Extraction

Write and submit a single *.sql* file containing the following queries, run directly against *ecommerce.db*. Each query must be preceded by a one-line comment explaining what it answers.

1. Total revenue, order count, and average order value by product category (net of discounts).
2. The top 20 customers by lifetime spend, including their city and signup date, using a JOIN across customers, orders, and order_items.
3. Month-over-month revenue trend for the last 24 months using a window function (e.g., LAG or a running total).
4. For each product category, the return rate (share of order_items with negative quantity) – written as a subquery or CTE.
5. Customers who have placed orders in every one of the last 3 calendar quarters (a query requiring HAVING with a COUNT of distinct periods).
6. The 10 products with the highest average review rating that also have at least 15 reviews (JOIN + GROUP BY + HAVING).
7. Average session duration and pages viewed by device type, restricted to customers who made at least one purchase (subquery/EXISTS).
8. A RANK() or DENSE_RANK() query ranking products by revenue within each category.
9. Payment-method mix (share of orders) split by country.
10. One query of your own design that answers a question you think leadership should care about – justify why in a comment.

Load the results of at least four of these queries into pandas (via *sqlite3* or *pandas.read_sql*) for use in later phases – don't just paste query outputs; keep them connected to your Python pipeline.

4.2 Phase 2 – Data Cleaning & Integration with Pandas

Using pandas, reconcile the three data sources (SQL extracts, legacy_customers_export.csv, product_catalog_2024.csv) into clean, analysis-ready DataFrames. Required tasks:

- Standardize the legacy CSV's four inconsistent date formats into a single datetime dtype.
- Detect and resolve duplicate/near-duplicate customer records (exact and fuzzy – e.g., same email, different name casing).
- Decide and justify a policy for missing values in age, city, gender, and review_text (drop vs. impute vs. flag) – do not silently fillna(0) everywhere.

- Identify and handle price outliers in products using an IQR or z-score rule; explain your threshold choice.
- Merge `product_catalog_2024.csv` with the products table on the appropriate keys, reconcile column-name/casing differences, and explicitly report which SKUs exist in only one source.
- Remove exact-duplicate `order_items` rows and decide how to treat negative quantities (returns) in revenue calculations.
- Reshape data at least once using `pivot_table` or `melt` (e.g., a category $\times$ month revenue matrix).
- Produce one time-series analysis using `resample` or `groupby` on a datetime index (e.g., weekly active customers).

Deliverable for this phase: a documented Jupyter notebook (or .py module) that takes the raw files as input and produces clean, saved DataFrames (e.g., as .parquet or .csv) that Phases 3 and 4 read from – your pipeline should be re-runnable end to end.

4.3 Phase 3 – Numerical Analysis with NumPy

Pandas convenience methods are not allowed for this phase – implement the following directly on NumPy arrays so you demonstrate the underlying computation. Show your formulas, not just library calls.

- **RFM Segmentation:** Bucket customers on Recency, Frequency, and Monetary value; compute the three scores as raw NumPy arrays (not via `pandas.qcut`) and combine them into a single segmentation score you define.
- **Similarity / Recommendation:** Build a product-similarity or customer-similarity matrix using cosine similarity computed with NumPy dot products / norms (**no library shortcut such as `sklearn's cosine_similarity`**). Recommend 3 products for 5 sample customers.
- **Regression via Normal Equation:** Fit a simple linear model (e.g., predicting monthly revenue from month index, or order value from discount level) using the normal equation $B = (X^T X)^{-1} X^T y$ implemented with `np.linalg`, not a fitted library model. Report R^2 computed manually.
- **Monte Carlo Simulation:** Run a Monte Carlo simulation ($\geq 5,000$ trials) using NumPy's random module to estimate the probability of stocking out for 3 chosen products under demand uncertainty; report a confidence interval.

For each routine, include in your report: the mathematical formula, a short explanation of what it measures, and a sanity check against a trivial case or a library function (used only to verify – not to compute your final answer).

4.4 Phase 4 – Business Insights & Visualization

Using your cleaned data and NumPy routines, answer the following with at least one supporting chart each (matplotlib or seaborn):

1. Which customer segment (from your RFM scores) generates the most revenue, and what distinguishes it demographically?
2. Is there seasonality in overall revenue? Support with a decomposition or rolling-average chart.
3. Which category has the highest effective margin after accounting for discounts and returns?
4. Do higher review ratings correlate with repeat-purchase behavior? Quantify the relationship.
5. Which device type/country combination shows the highest engagement-to-purchase conversion?
6. Based on your regression model, what is the projected revenue for the next 2 months, with an uncertainty range?

7. Which 5 products are the strongest stockout risks based on your Monte Carlo simulation, and what reorder point would you recommend?
8. What is one data-quality finding from Phase 2 that materially changed a business conclusion (i.e., the answer would have been wrong on the raw data)?

4.5 Phase 5 – Reusable Pipeline (Stretch, Recommended)

Package Phases 1-4 into a single re-runnable pipeline (a small set of .py modules or a Makefile/notebook sequence) that goes from the raw files to your final tables and figures with one command. This is not mandatory for full marks but is required for the “Exceeds Expectations” rubric tier, and it is exactly what you would need to hand this project off to a colleague or run it again next quarter.

5. Deliverables

- [queries.sql](#) – All SQL from Phase 1, runnable against the provided ecommerce.db.
- [analysis.ipynb](#) (or [.py modules + a driver notebook](#)) – Cleaning, integration, EDA, NumPy routines, and chart generation, organized in clearly labeled sections matching Phases 1-4.
- [/data/processed/](#) – A cleaned, saved dataset ready for reuse (e.g., clean_orders.csv, clean_customers.csv).
- [report.pdf](#) – Your written report, minimum 30 pages (see Section 6), submitted as PDF.
- [executive_summary.pdf](#) – A one-page summary of results for a non-technical executive audience (can double as your report’s executive summary, exported separately).
- [README.md](#) – A short (150-300 word) statement of what each team member contributed, if working in a team.
- **Please Submit all above as a GitHub repository By August 2nd 2026**

6. Written Report Requirements (Minimum 30 Pages)

The report is a primary deliverable, not an afterthought bolted onto the notebook. It should read as a standalone analytical document – assume the reader will not open your code. Minimum 30 pages, 11-12pt font, standard margins, excluding the title page and table of contents. The suggested section-by-section page budget below is a floor, not a ceiling: go longer where the analysis warrants it, but do not pad with whitespace, oversized figures, or repeated boilerplate to hit the count.

Section	Pages	Must include
Executive Summary	1-2	Top 5 findings and recommendations, written for a non-technical reader
Introduction & Business Context	1-2	The 3 business questions from Section 2, restated in your own words, and why they matter
Data Description	2-3	Schema/ERD, data sources, scale, and known limitations
SQL Extraction & Methodology	3-4	Each of the 10 queries with the query itself, its result (table/chart), and a 2-3 sentence interpretation
Data Cleaning & Integration	5-6	Every issue found, the rule used to fix it, before/after evidence (row counts, distributions), and what you chose NOT to fix and why
NumPy Analytical Methods	6-8	Formulas, derivations/explanations, implementation notes, and validation against a sanity check for each of the 4 routines
Findings & Insights	6-8	All 8 business questions from Phase 4, each with a chart, the numeric result, and a business interpretation
Limitations & Threats to Validity	1-2	What could make your conclusions wrong (sampling, missing-data handling, model assumptions)
Recommendations & Next Steps	1-2	3-5 concrete, prioritized actions UrbanCart should take
Appendices	3-5	Full SQL listing, additional charts, data dictionary, individual contribution statement
Total	30+	